

Goldman Sachs India Hackathon

Previous Year Questions

Watch Video Goldman Sachs: [Amisha Aggarwal](#)

Texas Instruments (Apply now) : [Amisha Aggarwal](#)

CS Track

Problem Statement

The primary goal is to develop a robust and efficient system that can accurately decode Code 39 barcode information from digital images, assuming relatively good image quality and lighting. Your task is to accurately identify and locate Code 39 barcode in an image, minimizing false positives and false negatives and output the decoded data.

Sample image below



Barcode Parsing

Output : GS2025

Input Format**Guidelines**

1. First line in the standard input represents the image to be used eg. image1.jpg

Sample Input

image2.jpg

Reading the Input

You can read the input as standard input.

To load the image, you can load it as following:

Python

```
//read from standard input filename = input() // get file name
os.path.isfile(filename):
# Open the file
with open(filename, 'r') as file:
...
```

Java

```
import java.io.File;
...
// read from standard input scanner scanner = new Scanner(System.in); String query =
scanner.nextLine(); // Get the image file
File file = new File("image1.jpg");
```

Constraints**Image constraints**

1. All images are in jpg

Output Format**Guidelines**

- Please output only the extracted Code 39 barcode data (the decoded string) in standard output. The symbology type is Code 39.

Sample Output

GS2025

Sample Input 0

image2.jpg

Sample Output 0

GS2025

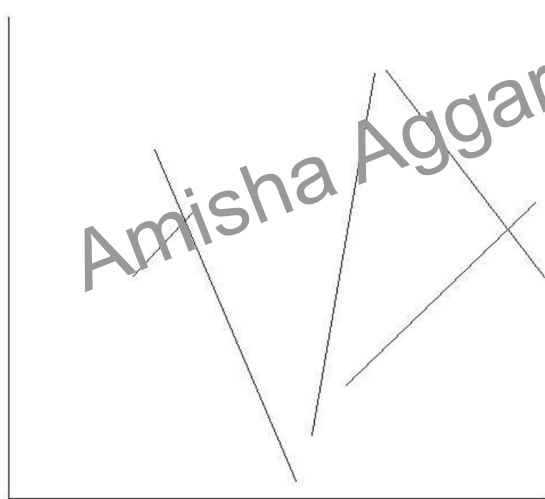
[Link](#)

Automated Intersection point detection in 2D Line Graphs

Problem Statement

The analysis of graphical data is a common task in various fields, including finance, economics, and data science. A fundamental operation in graph analysis is the identification and counting of intersection points between lines. Manually identifying these points can be time-consuming and prone to error, especially with complex graphs. Your task is to develop an algorithm and automate the process of accurately detecting and counting intersection points in 2D line graph images.

Sample image where there are 2 intersection points -



[Link](#)

Input Format

Guidelines

1. First line in the standard input represents the image to be used eg. image1.jpg

Bit Happens

Problem: Bit Happens

You are a digital forensics expert tasked with uncovering secrets buried deep within obscure file formats. Each case challenges you to decode hidden patterns, analyze packet captures, and simulate the execution of enigmatic bytecode left behind by an unknown system.

Each test case consists of a packet capture file and a **MysticLang** bytecode file. However, there is only **one shared bitmap file**, which may contain clues relevant to all cases.

The base64 encoded data of the shared bitmap file **common.bmp** is shown below

```
Qk02AwAAAAAAAAADYAAAAoAAAAEAAAAAAAAABABgAAAAAAAAADAAATCwAAEwsAAAAAAAAAAAAzOvv+fL9+Mz02vLa1s/Z
20H8ysz109Hw8u/R1+nS+uva80bhzufd2tzW09DT5PPiyObN+fzK90j14tbpztvJ7OP/9+DZ8dH7trn+dPx29zU3fV
2fn0ztT02ejx2/j12ur1/tDoy8jz3/bS6PzQ+OzSyuHY//Xr/sr30NLf/erM49re+NTv8PHs0u/k6dDv4N0Zz/L4vz2
49PN4tjh3/br4uHQ6ujn1tzip/vHu//Tt+cjz9eXL003X/PDS8Pn2/PDZ6uHr4+vL9/3Vyn16/bS6PzQ+OzSyuHY//Xr/sr30NLf/erM49re+NTv8PHs0u/k6dDv4N0Zz/L4vz2
68jl/97m3fr9+vPizNzk08rq/Org+/fwl+T0+d/zle/e39fV+u/K1+jb2PzLzM7w6tz38P/PF2/vn9vnb2P/PZNLN
80T19PHL2ubL6P/51tdk9tDz30/n79D0//W88741fj11/jR7+3t1dnuyNPr70Hj7M3x08P/0PwS16a/vu6d/690HW
0dPU0N3e4ufQ3uTR7Lvk5s3h//D3z/f0+uPd7vbl/v3m7eHj0uT34DnY+Nbs6eH69XX7P/PLyunkY&M3qy8305PrS
5Pj14wvS39v//ePM6Tv6e7Jy8/u5cNw9/Dt/8j14Pd9j9/Dt0uj52vXv9z2P/0CfLb5w/h20LI6+br6t3U803T9sRU
1Pf07dbS10f30/742mz93uvY1fLs60v83c3f00Lm7ePQ2d/0PwS16a/vu6d/690HW
0svt7PD02fxv3tf04+ba4uzjz93Q39/268v01NTV+0u24C/Q0PwS16a/vu6d/690HW
9+D16M/0792m703Y4f789ezV4ej77fPi5/XU5e+3d7Lecm8e110j599z3vufZ5tblYtTY0d/b5+vd583zztlr3vfr
+u/j2PzR0v3y0Ty4NLx8ej/69z55N/0PwS16a/vu6d/690HW
/k0r5+JLL+ff38/R383i79Dn+0Ty2ubS0e7S39n159Xs
```

Input

A single **base64-encoded string** representing a **JSON object** with the following structure:

JSON Structure (Before Base64 Encoding)

Sample input

```
{
  "n": 1,
  "data": [
    {
      "pcap": "bm90IGEdmFsaWQgcGcNhcA==",
      "myst": "bm90IGEdmFsaWQgbXlzdGxhbmcmcgaGV4Y29kZQ==",
      "memory_address": 2
    },
    ...
  ]
}
```

- `n` is the total number of test cases (length of `<data>` array)



Submissions: 1731

Max Score: 1000

Difficulty: Medium

Rate This Challenge:



[More](#)

[Link](#)

[CS] OMR sheet processing

locked

Problem

Submissions

Leaderboard

Discussions

Problem Statement

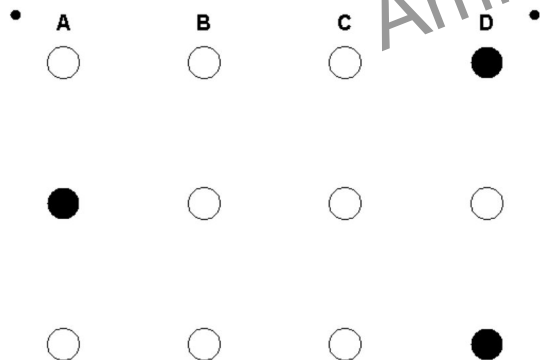
You are given an image representing a scanned OMR answer sheet. Each image contains a grid of circles, with each row representing one question and each column representing a possible answer (A, B, C, D). Some circles are filled to indicate the selected answer by the candidate.

Your task is to write a program that takes the image as input, along with an array containing the correct answers for each question and returns a score based on the number of correct answers. For each correct answer, score will be incremented by 1 point. Each question has only 1 correct answer.

The filled circles in the images can be either fully filled (360 degrees filled) or partially filled. A fully filled circle represents a definite answer, while a partially filled circle represents a probable answer. If a circle is filled more than 75% (270 degrees filled) of the area, it should be considered as a **definite** answer.

Sample OMR images

1. For the below image, when evaluating with the expected answer as D A D C, the score is evaluated as 4



Submissions: [622](#)

Max Score: 1000

Difficulty: Medium

Rate This Challenge:



[More](#)

[Link](#)

[CS] SQL Query Engine on JSON

locked

Problem

Submissions

Leaderboard

Discussions

Problem Statement

You are given 4 JSON files named **products**, **reviews**, **users**, **transactions** that contains information about various products sold by an e-commerce company, user information, reviews written by users and all transactions made on the products.

Your task is to develop a SQL query engine that can execute SQL queries on JSON data. The engine is expected to handle the following types of queries -

- SELECT
- WHERE
- JOIN (INNER type only)
- GROUP BY
- ORDER BY
- LIMIT
- Aggregate functions (COUNT(*), AVG, SUM, MIN, MAX)

The SQL query will be provided as input will always follow the syntax below. The way to read the syntax definition is - square brackets [] indicate optional presence of that element in the query and curly braces { } indicates mandatory where applicable. Pipe | separated values indicates any one of the options. ... indicates one or many such elements can exist.

```
`SELECT
    {table_alias | join_table_alias}.col_name AS column_alias,
    ...
FROM table table_alias
[ JOIN table join_table_alias on joinexpr [AND joinexpr ... ] ] ...
[WHERE expr [AND expr ... ] ]
[GROUP BY {table_alias.col_name, ... }]
ORDER BY {column_alias, ... } { ASC | DESC }
[LIMIT n]`
```

[Link](#)

[CS] Walk Again

locked

Problem

Submissions

Leaderboard

Discussions

Let G be an acyclic connected undirected graph with unit edge weights, where no node has more than two neighbors. Let A be the adjacency matrix of G . Define the matrix M as:

$M = I + 3A + 5A^2 + \dots + (2k-1)A^{k-1} + (2k+1)A^k$, where I is the identity matrix and k is the smallest positive integer such that all entries of M are positive.

Assuming A has the maximal possible order under these conditions, let $C0$ represent the minimal cost of an open walk that traverses all edges of G atleast once. The cost of a walk is determined by the sum of the edge weights as traversed.

Next, let $n = (C0 \bmod 17) + 3$, and consider an undirected graph H with n nodes labeled from 1 to n . For each i from 1 to n , define $w[i]$ as the probabilistic expected minimum value in an i element subset (i.e. size of subset equals i) of numbers 1, 2, ..., n .

In graph H :

- Every pair of nodes is directly connected except for the nodes with the smallest label 1 and the largest label n .
- The weight of the edge between any connected pair of nodes i and j is given by $\lceil w[i] \rceil + \lceil w[j] \rceil$.

Let $C1$ denote the minimal cost of a closed walk in H that traverses all its edges at least once.

We then embark on a final walk to emulate a convex polygon with each step corresponding to a side of the figure. Use the previously calculated $C1$ and figure out $C2$ which represents the number of different ways of dividing a convex polygon of $(C1 \bmod 107) + 3$ sides into potentially smaller polygons by adding zero or more non-intersecting diagonals. These diagonals must not meet at an interior point of the polygon but can share a common endpoint. For example, there're 3 ways to subdivide a given square ABCD by either choosing AC only, or choosing BD only or choosing nothing. Choosing AC and BD at the same time would run afoul of the constraints of non-intersecting diagonals. We define $C3$ as: $C3 = (C2 \bmod 10007) + 3$.

Task:

Given an input k , output the costs $C0$, $C1$ and $C3$.

Note: This problem statement has additional test cases that'll be run at the end of contest too. Scores may change as more test cases are added to the final version.

[Link](#)

Quant Track

Optimal Hedging Strategy

locked

Problem

Submissions

Leaderboard

Discussions

Description: Market making is a critical activity performed by financial institutions. These firms provide liquidity to financial markets by quoting bid and ask prices for various assets, enabling investors to buy and sell securities efficiently. Market makers generate revenue primarily through the bid-ask spread. They also profit from inventory management and taking positions based on market views.

However, market making activities inherently expose firms to market risk. Market risk refers to the potential for losses arising from changes in market conditions, such as fluctuations in asset prices, interest rates, or exchange rates etc. For an equities trader, this risk can manifest as potential losses due to adverse movements in the prices of the stocks they hold in their inventory or are obligated to trade.

To mitigate market risk, financial institutions employ various hedging strategies. Hedging involves taking offsetting positions in related assets to reduce the overall exposure to adverse market movements. The costs associated with hedging include transaction fees, bid-ask spread, and the capital cost of holding the hedges.

Objective: In this hackathon, you'll be stepping into the shoes of an equities trader facing market risk from an unhedged portfolio comprised of both equity stock and derivative products. You are given an unhedged portfolio's historical Profit and Loss (P&L) distribution, simulated using the last year's historical returns of the portfolio's constituent assets.

Ideally, hedging can be done with liquid equity stock to minimize transaction costs and ensure the ability to adjust positions quickly. Your objective is to identify a set of hedging stocks from a universe of publicly traded equity stocks and determine the quantity of each stock needed to minimize risk, while also considering the cost of implementing the hedge. These equity stocks have known risk profiles, represented by their historical daily returns, and a known hedging cost per share. The daily return of a stock is the percentage change in its price from one day to the next. For example, if a stock's price was 100 yesterday and 101 today, the daily return would be $(101-100)/100 = 0.01$ or 1%.

The effectiveness of your hedge will be evaluated based on the historical Value at Risk (VaR) of the hedged portfolio. Value at Risk (VaR) is a statistical measure of the potential loss in value of an asset or portfolio over a defined period for a given confidence level. $\text{VaR (confidence level)} = \text{The } (1 - \text{confidence level})\text{th percentile of the portfolio's return distribution.}$

Here's how you can calculate 95%ile VaR using Python:

```
import numpy as np
```

[Link](#)

Automated Market Making

locked

Problem

Submissions

Leaderboard

Discussions

Overview: You'll build an adaptive quoting strategy that mimics a market-maker in a volatile, real-time environment. Every timestamp, your code posts:

- bid_price
- ask_price

based on the L2 order book, recent public trades, and your current inventory. Maximize total PnL while keeping inventory within ± 20 units.

Data Provided:

1. orderbook_train.csv (training) - [Link](#)

- timestamped L2 snapshots: bid/ask prices & sizes at multiple levels.
- Columns: timestamp, bid_price_1, bid_size_1, ..., ask_price_2, ask_size_2, etc.

2. public_trades.csv (training) - [Link](#)

- timestamp, price, size, side (buy/sell)

3. orderbook_test.csv

4. public_trades_test.csv

Tick Size: All quotes must be multiples of 0.1. Any quote not on this grid is invalid and will incur hidden penalties.

PLEASE NOTE - PICK ONLY TOP **3000 rows** from all datasets for submission

Submission Files:

[Link](#)

Exotic Option Pricing using Monte Carlo Simulation

locked

Problem

Submissions

Leaderboard

Discussions

Imagine you're a quantitative analyst on an Equity Volatility trading desk at a major investment bank. Your client, a large hedge fund, has requested price quotes for custom European basket options with knock-out features. The basket is created on three hypothetical assets: DummyTechCorp (DTC), DummyFinCorp(DFC) and DummyEnergyCorp(DEC).

You are now tasked with building this pricing model and providing prices for all the options. Your submission will be rated based on the aggregate accuracy of the submitted prices.

Background:

Vanilla Options

An option is a financial contract that gives the buyer the right, but not the obligation, to buy or sell an asset at a specified price (called the strike price) on or before a specified date (called the maturity). Options are widely used for hedging, speculation, and portfolio management. We concern ourselves only with 'European Options' in this challenge. European options can only be exercised on the specified maturity date and not before.

Generally, there are two types of European vanilla options.

Call Options:

- A call option gives the holder the right to buy the underlying stock at the strike price K on the maturity date.
- The payoff of a call option is given by $\max(S(t) - K, 0)$ where $S(t)$ is the price of underlying stock on maturity date.

Put Options:

- A put option gives the holder the right to sell the underlying asset at the strike price K .
- The payoff of a put option is given by $\max(K - S(t), 0)$ where $S(t)$ is the price of underlying stock on maturity date.

[Link](#)

[QUANT] Option Pricing Model



Problem

Submissions

Leaderboard

Discussions

Description

Trading financial derivatives is one of the key functions for major investment banks.

A derivative is a contract between two or more parties whose value is based on an agreed-upon underlying financial asset (like a stock price) or set of assets (basket of stocks).

An example is a European call option on an equity stock(called the underlier of the option). This option gives the buyer the right (but not the obligation) to buy the underlying asset (equity stock in this case) at a fixed price (called strike) at a fixed time (called maturity/ expiry). In return, the option buyer would need to pay some price at the contract start date called the 'Option Price'. The buyer would exercise the option (i.e. buy the underlier at the strike price) only if the stock price at the expiration date is greater than the strike price.

Similarly, a European put option gives the buyer the right (but not the obligation) to sell the underlying asset (equity stock in this case) at a fixed price (called strike) at a fixed time (called maturity/ expiry). The buyer would exercise the option (i.e. buy the underlier at the strike price) only if the stock price at the expiration date is lower than the strike price.

Objective

The problem here deals with estimating the Option Price based on historical data available by employing a mathematical model. The underlier of the option is 1 unit of a publicly traded stock of a hypothetical company.

Modeling Introduction

Option Price majorly depends on following features:

- Type of the Option (Call or Put)
- Spot value of the underlier stock



Submissions: 330

Max Score: 1000

Difficulty: Medium

Rate This Challenge:



Rate

[Link](#)

[QUANT] Predict Closing Auction Volume

locked

Problem

Submissions

Leaderboard

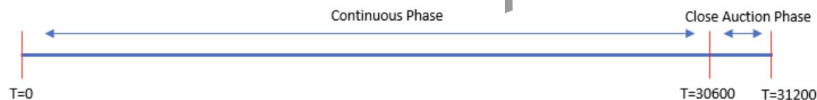
Discussions

Description

Close Auction Volume prediction throughout the trading period is essential for various trading strategies to make critical decisions.

Stock trades on exchange in two phases:

1. **Continuous phase** – stock trades in the phase from the start of day till Close Auction takes place. Trades continuously happen in this phase as buyers and sellers agree on a price. Volume of shares traded in this phase is called Order Book volume.
2. **Close Auction phase** – stock trades in this phase at the end of the trading. In this phase, large volume is traded at a single price through an auction, we call this volume as close auction volume.



[Link](#)

Objective

You have to develop a model which can predict close auction volume based on historic data provided.

Note: You should use training data to come up with a model that predicts the Close volume based on historic data.

Use the model that you have developed (based on training data), and with the input of test data, predict the Close volume for respective symbol_ids.

[QUANT] Manage your Stock Inventory



Problem

Submissions

Leaderboard

Discussions

On a daily basis you are broker facilitating buy and sell of stocks for different hedge funds to support their trading strategies. As a result you are required to manage the stock movements across different accounts.

Primary function is to deliver stocks to the clients based on their needs, any excess stocks that is present can be pledged in "triparty" accounts to be able to raise cash which can be used for client requirements.

Each movement of stock between accounts has a transaction cost associated with it. Lesser the number of movements lesser is the transaction cost. Additionally, each account belongs to a parent account. Aggregated transaction cost of the movement within the accounts of same parent account is always less than the transaction cost of the movement between accounts of different parent account, and hence should be always preferred. Another way to put this is cost of any number of transactions within the accounts of single parent account is less than cost of single transaction between 2 accounts that belong to different parent entity

Given the daily trading activity, devise an algorithm that will be able to effectively use the stocks to make sure the client demands are met on priority and maximizing the cash value of the asset, with minimum transaction cost possible. (Each row in the output is a transaction/movement.)

Input Format

First line is the number of stocks(n) and the next (n) lines are of format (*stock Id,price*) where *stock Id* is the stock identifier(string) and *price* is the stock price in \$USD.

2

P1,2.5

P2,1.25

Post above lines, you are given the number of accounts between which stocks can be transferred(n) and the

[Link](#)

Interview Experiences

- <https://www.geeksforgeeks.org/interview-experiences/juspay-interview-experience-for-sde-off-campus-2022/>
- <https://takeuforward.org/interview-experience/juspay-sde-interview-experience-set-1>